SCDAA Coursework 2025-26

Submit via Learn by Monday April 6th 2026 at noon

Overview

Your overall task will be to implement a numerical algorithm for solving stochastic control problems using policy iteration combined with the "deep Galerkin method" for solving a linear PDE.

This will be broken into steps which a) give you credit for completion and b) allow you to check your progress.

1. Create Python code for solving the LQR problem in 2 spatial and 2 control dimensions, see Section 1. We will need this to check our policy iteration algorithm produces correct answers in the end.

2. Train a neural network for the optimal Markovian control from the previous step (supervised learning), see Section 2 and train a different neural network for the value function using the value functions from the previous step. This will verify that the networks are "rich enough" and can provide good approximations.

3. Implement the "deep Galerkin method" for solving a linear PDE, see Section 3.

4. Implement the policy iteration, see Section 4.

You will be expected to submit a PDF report on what you did on Learn by the deadline. In the PDF provide a link to an online `git` repository where your code can be found. It should contain a `README.md` telling me how to run the code to reproduce any output (graphic / table) you include in the PDF. I will use the code from the latest commit made prior to the deadline.

Use `numpy`, `scipy`, `matplotlib` and `torch` and all the functions they contain but do not use other libraries. You will be working in groups of three[1] and the submitted PDF and `git` repo `README.md` should state your names, student numbers and individual contribution fractions.[2] You need to work together on each of the tasks at least to the extent that you can use code created in one task for the next one. There are 22 marks in total on this assignment but it is *marked out of* 20. Think of the extra two marks as a bonus.

Since most of you probably haven't used `torch` before ask Chat GPT for help whenever you need. You can access Chat GPT using `https://chat.openai.com` (you'll need to sign up).

[1] Self-organise. One person from each group should send `Gemma.Aitchison@ed.ac.uk` an email (Cc'ing the other members) with the subject `SCDAA 2026 CW group` stating names and student numbers of group members.

[2] The default is that you contributed equally i.e. $\frac{1}{3}$ each. If one of you feels they contributed more then they have to agree with the others about how you split this (e.g. $\frac{1}{2}$, $\frac{1}{4}$, $\frac{1}{4}$). If you cannot agree speak with the course organiser.

Please note that a submission that doesn't demonstrate why things work (e.g. test examples, convergence plots etc.) will get very few marks even if you have code that correctly solves the exercises.

It probably shouldn't need saying at this stage of your studies but any plot must have a **title**, **axis scales** and **axis labels**. Before submitting a final plot, think what kind of convergence you expect and use a logarithmic scales if appropriate. For example in Exercise 1.2 you should know what rates of convergence to expect from your NPMC course.

1 Linear quadratic regulator

See Example 4.13 in [3]. We consider

$$dX_s = [HX_s + M\alpha_s]\, ds + \sigma dW_s\,, s \in [t,T]\,, X_t = x\,. \tag{1}$$

Our aim is to **minimize**

$$J^\alpha(t,x) := \mathbb{E}^{t,x}\left[\int_t^T (X_s^\top C X_s + \alpha^\top D\alpha_s)\, ds + X_T^\top R X_T\right]\,,$$

where[3] $C \geq 0$, $R \geq 0$ and $D > 0$, are given and deterministic and we will assume 2×2.

The value function is $v(t,x) := \inf_\alpha J^\alpha(t,x)$. We know how to solve the Bellman PDE to obtain that

$$v(t,x) = x^\top S(t)x + \int_t^T \mathrm{tr}(\sigma\sigma^\top S(r))\, dr\,,$$

where S is the solution of the Ricatti ODE:

$$S'(r) = -2H^\top S(r) + S(r)MD^{-1}M^\top S(r) - C\,,\ \ r \in [t,T]\,,\ \ S(T) = R\,.$$

Note that solution takes values in the space of 2×2 matrices. The optimal Markov control is

$$a(t,x) = -D^{-1}M^\top S(t)x\,.$$

Exercise 1.1 (Solving LQR using from Ricatti ODE, 1 mark)**.**
Write a class which:

i) Can be initialised with the matrices specifying the LQR problem and $T > 0$.

ii) Has a method which will solve (approximate) the associated Ricatti ODE on a time grid which is an input (numpy array or torch tensor).

iii) Has a method that, given one torch tensor of dimension batch size (for time) and another torch tensor of dimension batch size $\times$ 1 $\times$ 2 (for space), will return a torch tensor of dimension batch size $\times$ 1 with entries being the control problem value $v(t,x)$ for each t,x in the batch (for x two dimensional).

iv) Has a method that, given one torch tensor of dimension batch size (for time) and another torch tensor of dimension batch size $\times$ 1 $\times$ 2 (for space), will return a torch tensor of dimension batch size $\times$ 2 with entries being the Markov control function for each t,x in the batch (for x two dimensional).

[3] For any matrix $\mathcal{M}$ we will write $\mathcal{M} \geq 0$ to denote positive definite matrices i.e. matrices such that for any $\xi \in \mathbb{R}^n$ we have $\xi^\top \mathcal{M}\xi \geq 0$. Similarly $\mathcal{M} > 0$ denotes strictly positive definite matrices i.e. those such that for any $\xi \in \mathbb{R}^n$, $\xi \neq 0$, we have $\xi^\top \mathcal{M}\xi > 0$.

Exercise 1.2 (LQR MC checks, 6 marks)**.** Run a Monte Carlo simulation of the system with the optimal solution you have obtained and ensure that you're converging to the optimal value function you obtained in Exercise 1.1. In particular:

1. Define how you'll measure the error and justify your choice.
2. With number of Monte Carlo samples large (e.g. 10^5) vary the number of time steps in your simulation, take $1, 10, 50, 100, 500, 1000, 5000$ and plot the error as a log-log plot.
3. With a number of time steps large e.g. 5000 vary the number of Monte-Carlo samples, take $10, 50, 10^2, 5 \cdot 10^2, 10^3, 5 \cdot 10^3, 10^4, 5 \cdot 10^4, 10^5$ and plot the error as a log-log plot.

Hint: Let $a = a(t, x)$ denote the optimal control from Exercise 1.1. Once you've plugged this in (1) you get

$$dX_s = [HX_s + Ma(s, X_s)]\, ds + \sigma dW_s\,, s \in [t, T]\,, X_t = x\,. \tag{2}$$

To do a time discretisation fix $N \in \mathbb{N}$ (number of time steps); let $\tau := T/N$ be the time step. Assume from now that you'd only possible want to start the SDE at times $t_k = k\tau$ for some $k = 0, 1, \ldots, N$. You have a choice of (at least) two schemes since you know $a(t, x) = -D^{-1}M^\top S(t)x$.

Explicit:

$$X^N_{t_{n+1}} = X^N_{t_n} + \tau\,[HX^N_{t_n} - MD^{-1}M^\top S(t_n)X^N_{t_n})] + \sigma(W_{t_{n+1}} - W_{t_n})\,,\ \ n = k, \ldots, N\,,\ \ X^N_{t_k} = x\,.$$

Implicit:

$$X^N_{t_{n+1}} = X^N_{t_n} + \tau\,[HX^N_{t_{n+1}} - MD^{-1}M^\top S(t_{n+1})X^N_{t_{n+1}})] + \sigma(W_{t_{n+1}} - W_{t_n})\,,\ \ n = k, \ldots, N\,,\ \ X^N_{t_k} = x\,.$$

The implicit scheme will require you solve a linear system at each time step.

Warning: In Exercise 1.1 you're using a numerical method (perhaps your own, perhaps you chose to call Python method `solve_ivp` from `scipy.integrate`) to solve the Ricatti ODE. At some point the Monte-Carlo simulation may be more accurate than the numerical solver for the Ricatti ODE which means you won't see the desired convergence *unless you increase the accuracy of the solver used to solve the Ricatti ODE.*

2 Supervised learning, checking the NNs are good enough

For our purposes a neural network is a parametric function that depends on an input, say $x \in \mathbb{R}^d$ and parameters, say $\theta \in \mathbb{R}^p$ and takes values in say $\mathbb{R}^{d'}$ and we'll write $\varphi = \varphi(x; \theta)$.

An example of one hidden layer neural network is

$$\varphi(x; \theta) = \varphi(x; \alpha^{(1)}, \alpha^{(2)}, \beta^{(1)}, \beta^{(2)}) = \alpha^{(1)}\psi(\alpha^{(2)}x + \beta^{(2)}) + \beta^{(1)}\,,$$

where ψ is an activation function that is getting applied component-wise, $\alpha^{(2)}$ is a $h \times d$ matrix, $\beta^{(2)}$ is an h-dimensional vector, $\alpha^{(1)}$ is a $d' \times n$ matrix and $\beta^{(1)}$ is a d'-dimensional vector. In this case $\theta = \left(\alpha^{(1)}, \alpha^{(2)}, \beta^{(1)}, \beta^{(2)}\right)$ so that $p = h \times d + h + d' \times h + d'$ is the number of parameters or "weights" depending on the size of the hidden layer h.

The type of supervised learning task that is the most relevant for our purposes is the following. Our aim is to find neural network (NN) weights θ^* such that our NN $\varphi(\cdot;\theta^*)$ is a good approximation of some function f.

We get a training data set $\big(x^{(i)}, f(x^{(i)})\big)_{i=1}^{N_{\text{data}}}$ and search for θ^* by attempting to minimize

$$R(\theta) := \frac{1}{N_{\text{data}}} \sum_{i=1}^{N_{\text{data}}} \left|\varphi(x^{(i)};\theta) - f(x^{(i)})\right|^2$$

over $\theta \in \mathbb{R}^p$ by running (some variant) of a gradient descent algorithm i.e. from an initial guess of $\theta^{(0)}$ we update

$$\theta^{(k+1)} = \theta^{(k)} - \gamma \nabla_\theta R(\theta^{(k)})\,, \quad k = 0, 1, 2, \dots\,.$$

Exercise 2.1 (Supervised learning of value function v, 3 marks)**.** Take the neural network [1, `Net_DGM` class] with hidden layer size of 100.

With $T = 1$, use Exercise 1.1 to generate training data $v(t^{(i)}, x^{(i)})$ by sampling t uniformly on $[0, T]$ and x uniformly on $[-3, 3] \times [-3, 3]$.

Use the torch Adam optimizer to find network weights which give a good approximation of the value functions. Provide a plot of the training loss.

Exercise 2.2 (Supervised learning of Markov control a, 2 marks)**.** Carry out the same task as in Exercise 2.1 but for the Markov control from Exercise 1.1. Use the neural network [1, `FFN` class] with 2 hidden layers of size 100. Note that the output of the network must be 2-dimensional in this case as the function you're approximating returns a 2-dimensional vector. Provide a plot of the training loss.

3 Deep Galerkin approximation for a linear PDE

Consider the linear PDE where $\alpha = (1,1)^\top$ and H, M, C, D, R, σ are the matrices from Exercise (1.1).

$$\begin{aligned}\partial_t u + \tfrac{1}{2}\mathrm{tr}(\sigma\sigma^\top \partial_{xx} u) + (\partial_x u)^\top H x + (\partial_x u)^\top M\alpha + x^\top C x + \alpha^\top D\alpha = 0 \ \text{ on } \ [0,T) \times \mathbb{R}^2\,,\\ u(T,x) = x^\top R x \ \text{ on } \ \mathbb{R}^2\,.\end{aligned}$$

This is the linearisation of the Bellman PDE resulting from taking the constant control $\alpha = (1,1)^\top$ regardless of the state of the system.

The deep Galerkin method replaces u by a neural network approximation $u(\cdot,\cdot;\theta)$, chooses random points from the problem domain $(t^{(i)}, x^{(i)})$ with $i = 1, \dots, N_{\text{batch}}$ and them aims to minimize

$$\begin{aligned}R(\theta) &:= R_{\text{eqn}}(\theta) + R_{\text{boundary}}(\theta)\\ &= \frac{1}{N_{\text{batch}}} \sum_{i=1}^{N_{\text{batch}}} \Big| \partial u(t^{(i)}, x^{(i)};\theta) + \tfrac{1}{2}\mathrm{tr}(\sigma\sigma^\top \partial_{xx} u(t^{(i)}, x^{(i)};\theta))\\ &\qquad + (\partial_x u(t^{(i)}, x^{(i)};\theta))^\top H x^{(i)} + (\partial_x u(t^{(i)}, x^{(i)};\theta))^\top M\alpha + (x^{(i)})^\top C x^{(i)} + \alpha^\top D\alpha \Big|^2\\ &\qquad + \frac{1}{N_{\text{batch}}} \sum_{i=1}^{N_{\text{batch}}} \left| u(T, x^{(i)};\theta) - (x^{(i)})^\top R x^{(i)} \right|^2\end{aligned}$$

over θ in $\mathbb{R}^p$. Since the RHS of the PDE we're solving is zero if we can get $R(\theta)$ to be equal to zero then we have a good approximation of the solution.

Exercise 3.1 (Deep galerkin, Linear PDE, 5 marks)**.** Implement the Deep Galerkin method for the above linear PDE. You can use [1, `pde_BlackScholes_exchange_dgm.py`] or [2] as starting point / inspiration.

Provide: plot of the training loss and also (perhaps not at every learning step but at reasonable intervals) error against the Monte Carlo solution which you will adapt from Exercise 1.2 by replacing the optimal control with the constant α.

4 Policy iteration with DGM

Recall the policy iteration / improvement algorithm PIA [3, Section 4.4]. We will approximate the value function v by $v(\cdot,\cdot;\theta_{\text{val}})$ and the Marko controls a by $a(\cdot,\cdot;\theta_{\text{act}})$

At each iteration we will:

i) Given a Markov control function a approximated by $a(\cdot,\cdot;\theta_{\text{act}})$ we will need to solve

$$\partial u_t + \tfrac{1}{2}\text{tr}(\sigma\sigma^\top \partial_{xx} u) + (\partial_x u)^\top H x + (\partial_x u)^\top M a + x^\top C x + a^\top D a = 0 \text{ on } [0,T)\times\mathbb{R}^2\,,$$
$$u(T,x) = x^\top R x \text{ on } \mathbb{R}^2\,.$$

We already know how to solve the linear PDE: we'll just adapt Exercise 3.1 by replacing the constant α with output of $a(\cdot,\cdot;\theta_{\text{act}})$. This will give as an update of θ_{val}

ii) Now with θ_{val} fixed we want to update θ_{act} so that we minimize the Hamiltonian:

$$\begin{aligned} H(\theta_{\text{act}}) = &\frac{1}{N_{\text{batch}}}\sum_{i=1}^{N_{\text{batch}}}\Big[(\partial_x v(t^{(i)},x^{(i)};\theta_{\text{val}}))^\top H x^{(i)} \\ &+ (\partial_x v(t^{(i)},x^{(i)};\theta_{\text{val}}))^\top M a(t^{(i)},x^{(i)};\theta_{\text{act}}) \\ &+ (x^{(i)})^\top C x^{(i)} + a(t^{(i)},x^{(i)};\theta_{\text{act}})^\top D a(t^{(i)},x^{(i)};\theta_{\text{act}})\Big]\,. \end{aligned}$$

Note that here we're deliberately *not* working with mean-square-error. The algorithm simply minimizes the Hamiltonian (and most likely not to 0).

Exercise 4.1 (PIA with DGM, 5 marks)**.** Implement the policy iteration algorithm and check that it converges to the correct solution by comparing with the optimal value and actions from Exercise 1.1.

References

[1] M. Sabate-Vidales, Deep-PDE-Solvers, *Github project*, `https://github.com/msabvid/Deep-PDE-Solvers`, 2021.

[2] C. Jiang, Deep Galerkin Method, *Github project*, `https://github.com/EurasianEagleOwl/DeepGalerkinMethod`, 2022.

[3] G. dos Reis, J. Li, and D. Šiška. *Stochastic Control and Dynamic Asset Allocation lecture notes*. 2026.